

General advice

- Python is probably the best language to code in for LeetCode type problems. You can focus more on the algorithm rather than the syntax details of the programming language because Python has simple syntax.
- **Take notes of what the interviewer is saying.** It's not guaranteed that they'll put the question and test cases up on the screen, and it can be a bad look if you ask questions that you would already know the answer to if you were paying attention.
- **Ask the interviewer questions.** "What is the recommended time complexity?" "What is the recommended space complexity?" Ask about assumptions about the input (e.g. is it already sorted?). It's okay to ask the interviewer about their thoughts on two different approaches/algorithms to implement a solution.
- If you're given time/space complexity recommendations/constraints, use them to give you hints about what data structures or algorithms might be the best for solving the problem.
- You can start off defining variables or functions that you can easily determine or know you will need to define. Examples include the variables that are used in time/space complexity recommendations/constraints and the value you need to return.
- **Give variables descriptive names** (e.g. `for carIndex in range(len(cars))` rather than `for i in range(len(c))`), or else write a comment explaining what values a variable holds if it would be hard to come up with a decently short, descriptive name for it.
- Don't worry about defining too many or unnecessary variables. The added readability is worth the negligible inefficiency.
- **Write comments** to help yourself and the interviewer understand what your code is doing and your thought process.
- Don't hesitate to **define helper functions**. If you do, write comments about requirements of the input, what value(s) it returns, if any, and what side-effects it has, if any.
- At least initially, focus more on writing a correct algorithm rather than an efficient one.
- **Write test cases**, and draw pictures of them to help you think through how you would solve the given problem on those test cases (or how your code handles those test cases). Don't forget edge cases.
- **Practice, practice, practice!** LeetCode is hard, so don't feel bad if you can't solve a problem the first time you see it. But get comfortable with using the various data structures/common techniques, and you should become better at recognizing patterns so that you can solve problems you've never seen before more easily. The LeetCode 150 is a good place to start.

LeetCode topics

Topic	Used for...	Examples
Arrays & Hashing	Quickly checking whether something exists, counting occurrences, or grouping items, when you can afford extra memory to avoid repeated scanning	Checking for duplicates, counting frequencies, grouping items by key
Two Pointers	Scanning sorted or orderable data from both ends (or at different speeds) to avoid nested loops when comparing or narrowing a range	Finding a pair that sums to a target in a sorted array, detecting palindromes, merging two sorted lists
Stack	Tracking and resolving the most recently opened or added item first, especially with nested or reversible structure	Validating matched brackets, evaluating expressions, finding the next greater element
Binary Search	Narrowing down an answer in a sorted or monotonic space in logarithmic time, instead of scanning linearly	Searching a sorted array, finding the minimum in a rotated sorted array
Sliding Window	Tracking a running property (sum, count, distinct elements) over a contiguous range, updating incrementally as the range grows or shrinks instead of recomputing it	Longest substring without repeating characters, maximum sum subarray of fixed size, minimum window containing all target characters
Linked List	Frequent insertion or deletion at arbitrary points without shifting other elements, or reordering or merging	Reversing a linked list, detecting a cycle, merging K sorted lists
Trees	Representing and navigating hierarchical or ancestor/descendant relationships, or getting ordered, efficient search and insert	Computing tree depth, level order traversal
Tries	Efficiently storing and querying strings by shared prefixes	Implementing trie, prefix search
Backtracking	Systematically exploring every valid combination of choices while pruning branches as soon as they become invalid	Generating permutations/subsets, N-Queens
Heap/Priority Queue	Repeatedly accessing the current min or max element from a changing set, without needing the rest fully sorted	Finding the kth largest element in an array
Graphs	Modeling connections between entities and answering questions about reachability, connectivity, or ordering between them	Finding the shortest path in an unweighted graph, number of islands, topological sort
1-D Dynamic Programming	Building up an optimal answer over a single sequence or index, where each step reuses a small number of already-solved smaller cases	Climbing stairs, coin change, longest increasing subsequence
Intervals	Reasoning about overlap, coverage, or scheduling among ranges with a start and end	Merging overlapping intervals, meeting room scheduling
Greedy	Making a sequence of locally optimal choices, when you can rely on those choices adding up to the globally optimal outcome without backtracking	Maximum sum subarray, jump game
Advanced Graphs	Finding optimal paths or structures in graphs where edges carry weights or capacities, beyond what plain BFS/DFS can answer	Shortest path with weighted edges (Dijkstra), minimum spanning tree (Prim's/Kruskal's)
2-D Dynamic Programming	Building up an optimal answer that depends on two indices at once, typically when comparing two sequences or moving through a grid	Unique paths through a grid, regular expression matching
Bit Manipulation	Working directly with binary representations for speed and low memory, such as flags, sets, or parity-based logic	Counting 1 bits, summing two integers without + or -
Math & Geometry	Problems that hinge on number-theoretic or spatial/coordinate reasoning rather than a general traversal or search pattern	Computing GCD/LCM, checking primality, rotating a matrix in place

Time complexity

- $O(1)$ - index into list, retrieve from/insert into set, retrieve from/insert key into dictionary
- $O(\log(n))$ - binary search (or otherwise halving search space each iteration), insert into/pop from priority queue
- $O(n)$ - iterating through list or linked list
- $O(n * \log(n))$ - sorting, n insertions into a priority queue
- $O(n^2)$ - nested for loop, compare each element to every other element (i.e examine all pairs of elements)
- $O(2^n)$ - generating all subsets of n elements
- $O(n!)$ - generating all permutations of n elements

Space complexity

- $O(1)$ - two pointers, sliding window, constant number of variables
- $O(\log(n))$ - recursion on balanced binary tree
- $O(n)$ - list
- $O(n * \log(n))$ - mergesort
- $O(n^2)$ - matrices (list of lists)
- $O(2^n)$ - storing all subsets
- $O(n!)$ - storing all permutations

Data structures in Python

List (also called array; can use to implement a stack) Initialize with: <code>l = [], l = list()</code>		
Append value v to the back of l	<code>l.append(v)</code>	$O(1)$
Retrieve the element at index i of l	<code>l[i]</code>	$O(1)$
Pop from the back of l	<code>l.pop()</code>	$O(1)$

Set Initialize with: <code>s = set(), s = set(l: List)</code>		
Add to value v to s	<code>s.add(v)</code>	$O(1)$
Check if value v in s	<code>v in s</code>	$O(1)$
Remove value v from s	<code>s.remove(v)</code> <code>s.discard(v)</code>	$O(1)$

Dictionary (also called hash map) Keys can be anything that is hashable- strings, ints, user-defined objects. Lists are not hashable because they're mutable, but tuples are immutable and hashable. Initialize with: <code>d = {}, d = dict()</code>		
Assign value v to key k	<code>d[k] = v</code>	$O(1)$
Check if key k in d	<code>k in d</code>	$O(1)$
Retrieve the value with key k	<code>d[k]</code> <code>d.get(k, fallbackValue)</code>	$O(1)$
Delete key k from d	<code>del d[k]</code> <code>d.pop(k)</code>	$O(1)$

Queue (can use to implement a stack. Note deque is an abbreviation for double-ended queue and is pronounced "deck") Initialize with: <code>from collections import deque</code> <code>q = deque()</code>		
Append value v to the back of q	<code>q.append(v)</code>	$O(1)$
Append value v to the front of q	<code>q.appendleft(v)</code>	$O(1)$
Retrieve the element at the back of q	<code>q[-1]</code>	$O(1)$
Retrieve the element at the front of q	<code>q[0]</code>	$O(1)$
Pop from the back of q	<code>q.pop()</code>	$O(1)$
Pop from the front of q	<code>q.popleft()</code>	$O(1)$

Priority Queue (also called heap; Python's implementation is a min-heap)

Initialize with:

```
import heapq
```

```
pq = []
```

Note: rather than using pairs with explicit priorities, you may also use an instance of a class with the `__lt__(self, other)` method defined to determine which of two different instances has the higher priority.

Make pq heap-ordered	<code>heapq.heapify(pq)</code>	$O(n)$
Push value v with priority p into pq	<code>heapq.heappush(pq, (p, v))</code>	$O(\log(n))$
Retrieve the (p, v) pair with the minimum p in pq	<code>pq[0]</code>	$O(1)$
Pop the (p, v) pair with the minimum p from pq	<code>heapq.heappop(pq)</code>	$O(\log(n))$

Disjoint Set (also called union-find)

Initialize with:

```
from scipy.cluster.hierarchy import DisjointSet
```

```
ds = DisjointSet()
```

Add value v to ds	<code>ds.add(v)</code>	$O(1)$
Merge (<i>union</i>) u and v 's subsets	<code>ds.merge(u, v)</code>	$O(1)$
Check whether u and v are in the same subset	<code>ds.connected(u, v)</code>	$O(1)$
Retrieve (<i>find</i>) the subset containing v	<code>ds.subset(v)</code>	$O(1)$
Retrieve all subsets	<code>ds.subsets()</code>	$O(1)$

User-defined data structures

Singly linked list

Definition:

```
class ListNode:
```

```
    def __init__(self, val=0, next=None):
```

```
        self.val = val
```

```
        self.next = next    # ListNode
```

Initialize with:

```
head = ListNode()
```

Doubly linked list

Definition:

```
class DoublyListNode:
```

```
    def __init__(self, val=0, prev=None, next=None):
```

```
        self.val = val
```

```
        self.prev = prev    # DoublyListNode
```

```
        self.next = next    # DoublyListNode
```

Initialize with:

```
head = tail = DoublyListNode()
```

Binary tree

Definition:

```
class TreeNode:
```

```
    def __init__(self, val=0, left=None, right=None):
```

```
        self.val = val
```

```
        self.left = left    # TreeNode
```

```
        self.right = right   # TreeNode
```

Initialize with:

```
root = TreeNode()
```

Prefix tree (also called trie)

Definition:

```
class TrieNode:
```

```
    def __init__(self):
```

```
        self.children = {}    # Dictionary mapping values to TrieNodes
```

```
        self.is_end = False
```

Initialize with:

```
root = TrieNode()
```

Graph

Definition:

```
class GraphNode:
```

```
    def __init__(self, val=0, neighbors=None):
```

```
        self.val = val
```

```
        self.neighbors = neighbors if neighbors is not None else []    # List of GraphNodes
```

Initialize with:

```
node = GraphNode()
```

Built-in functions

- `print(obj)` prints `obj` to the terminal
- `type(obj)` returns the type of `obj`
- `range(start?: int, stop: int, step?: int)` returns an iterable container from `start` up to but not including `stop` with `step` steps in between each element
- `len(obj)` returns the length of `obj`. For `str`, the number of characters in it; for `List` and `Set`, the number of items in it; for `Dictionary`, the number of keys in it
- `sorted(i, key?)` returns a sorted `List` of `i`'s elements, where `i` is an iterable container such as a `List`, `Set`, or `Dictionary`, leaving `i` unchanged
 - If `i` is a `Set`, it returns a sorted `List` of `i`'s values. If `i` is a `Dictionary`, it returns a sorted `List` of the `i`'s keys
 - `key` can be a lambda such as `key=lambda item: item[1]` or `key=lambda x: -x` or `key=lambda c: -ord(c)`. The lambda is applied to every value in `i` and sorts based on the return values of the lambda
- `max(i, key?)` returns the maximum element in `i`, where `i` is an iterable container such as a `List`, `Set`, or `Dictionary`
- `max(arg1, arg2, ..., key?)` returns the maximum argument among the provided arguments
- `min(...)` works similarly to `max()`
- `str(n: num)` returns `n` interpreted as a `str`, leaving `n` unchanged
- `int(s: str)` returns `s` interpreted as an `int`, leaving `s` unchanged
- `float(s: str)` converts `s` to a `float`, leaving `s` unchanged
 - `float('inf')` and `float('-inf')` can represent infinity and negative infinity respectively
- `ord(c)` returns the unicode code of `c`, where `c` is a `str` with only one character
- `abs(n: num)` returns the absolute value of `n`

Binary arithmetic operators

- `//` - floor division
- `%` - modulo
- `**` - exponentiation
- `&` - bitwise AND
- `|` - bitwise OR
- `^` - bitwise XOR
- `<<` - shift bits left
- `>>` - shift bits right
- `~` - bitwise NOT

Common methods/operators of built-in types

- `s[start:stop:step]` returns the substring of `s` starting at index `start` up to but not including index `stop` with `step` steps in between each character
 - `start`, `stop`, and `step` are all optional, and `step` can be negative
 - This also works with sequential containers that support indexing including `Lists` and `Tuples`
- `s.lower()` returns a lowercase copy of `str s`, leaving `s` unchanged
- `s.upper()` returns an uppercase copy of `str s`, leaving `s` unchanged
- `s.split(sep?: str)` returns a `List` splitting `str s` with the delimiter `sep`, or any whitespace if `sep` isn't provided
- `sep.join(i)` returns a `str` joining the elements of `i` with the delimiter `sep`, where `i` is an iterable container such as a `List` or `Set` containing only `strs`
- `l.sort(key?)` sorts `List l` in-place, modifying `l`
- `d.items()` returns an iterable container of key-value pair `Tuples` of `Dictionary d`

Example solutions

Given an unsorted array of integers `nums` and an integer `k`, return the `k`th largest element in the array.

```
import heapq
```

```
class Solution:
```

```
    def findKthLargest(self, nums: List[int], k: int) -> int:
        minHeap = []

        for num in nums:
            heapq.heappush(minHeap, num)

            if len(minHeap) > k:
                heapq.heappop(minHeap)

        return minHeap[0]
```

Given the root of a binary tree `root`, return the diameter of the tree.

```
class Solution:
```

```
    def diameterOfBinaryTree(self, root: Optional[TreeNode]) -> int:
        res = 0

        # Returns: Height of node
        # Effects: Updates res
        def dfs(node):
            if node is None:
                return 0

            leftHeight = dfs(node.left)
            rightHeight = dfs(node.right)
            nonlocal res
            res = max(res, leftHeight + rightHeight)

            return 1 + max(leftHeight, rightHeight)

        dfs(root)
        return res
```

Given an array `nums` of unique integers, return all the possible permutations. You may return the answer in any order.

```
class Solution:
```

```
    def permute(self, nums: List[int]) -> List[List[int]]:
        permutations = []

        def genPerms(curr, permLength):
            if permLength == len(nums):
                permutations.append(curr.copy())
                return

            for i in range(permLength, len(nums), 1):
                curr[permLength], curr[i] = curr[i], curr[permLength]
                genPerms(curr, permLength + 1)
                curr[permLength], curr[i] = curr[i], curr[permLength]

        genPerms(nums, 0)
        return permutations
```

Given an integer array `nums`, return the length of the longest strictly increasing subsequence.

```
class Solution:
```

```
    def lengthOfLIS(self, nums: List[int]) -> int:
        n = len(nums)
        # memo[i] is the length of the longest increasing subsequence
        # ending at index i
        memo = [1 for _ in range(len(nums))]
        maxLen = 1

        for i in range(1, n):
            for j in range(i):
                if nums[j] < nums[i]:
                    memo[i] = max(memo[i], memo[j] + 1)
                    maxLen = max(maxLen, memo[i])

        return maxLen
```